Sama nabil SE2

Lab one

Part A

Compare virtual machines and containers in terms of resource usage and architecture.

```
sama@DESKTOP-L3NT4QA:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.7Gi       462Mi       2.9Gi       3.5Mi       551Mi       3.3Gi
Swap:          1.0Gi          0B       1.0Gi
```

```
sama@DESKTOP-L3NT4QA:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd       1007G  1.3G  955G   1% /
rootfs          1.9G  2.7M  1.9G   1% /init
none            1.9G     0  1.9G   0% /dev/shm
none            1.9G  516K  1.9G   1% /run
none            1.9G     0  1.9G   0% /run/lock
none            1.9G   76K  1.9G   1% /run/user
tmpfs           384M   20K  384M   1% /run/user/1000
drivers         238G  181G   57G  77% /usr/lib/wsl/drivers
none            1.9G     0  1.9G   0% /usr/lib/wsl/lib
none            1.9G  4.0K  1.9G   1% /mnt/wsl
none            1.9G   76K  1.9G   1% /mnt/wslg/versions.txt
none            1.9G   76K  1.9G   1% /mnt/wslg/doc
none            1.9G     0  1.9G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
C:\             238G  181G   57G  77% /mnt/c
D:\             932G  304G  629G  33% /mnt/d
```

```
sama@DESKTOP-L3NT4QA:~$ ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.4  0.3  21964 12416 ?        Ss   22:20   0:01 /sbin/init
root           2  0.0  0.0   3120  1920 ?        Sl   22:20   0:00 /init
root           7  0.0  0.0   3136  1792 ?        Sl   22:20   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8
--pipe-fd 10 --log-truncate
root          53  0.0  0.3  42244 15360 ?        S<s  22:20   0:00 /usr/lib/systemd/systemd-journald
root         113  0.0  0.1  25276  6144 ?        Ss   22:20   0:00 /usr/lib/systemd/systemd-udevd
systemd+     123  0.0  0.3  21460 12288 ?        Ss   22:20   0:00 /usr/lib/systemd/systemd-resolved
systemd+     124  0.0  0.1  91028  7808 ?        Ssl  22:20   0:00 /usr/lib/systemd/systemd-timesyncd
root         177  0.0  0.0   4236  2560 ?        Ss   22:20   0:00 /usr/sbin/cron -f -P
message+     178  0.0  0.1   9520  4736 ?        Ss   22:20   0:00 @dbus-daemon --system --address=systemd: --nofork --n
opidfile --systemd-activation --syslog-only
```

Windows Subsystem for Linux (WSL2) was used as a lightweight virtualized environment due to limitations in Hyper-V availability. It provides a Linux kernel running on a virtualized layer, similar to a VM but with lower overhead compared to traditional virtual machines.

2)

```
Status: Downloaded newer image for ubuntu:latest
root@33de406dce50:/# free -h
               total        used        free      shared  buff/cache   available
Mem:           3.7Gi       811Mi       599Mi       4.6Mi       2.5Gi       3.0Gi
Swap:          1.0Gi          0B       1.0Gi
```

```
root@33de406dce50:/#  df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay        1007G  1.7G  955G   1% /
tmpfs            64M     0   64M   0% /dev
shm              64M     0   64M   0% /dev/shm
/dev/sde       1007G  1.7G  955G   1% /etc/hosts
tmpfs           1.9G     0  1.9G   0% /proc/acpi
tmpfs           1.9G     0  1.9G   0% /proc/scsi
tmpfs           1.9G     0  1.9G   0% /sys/firmware
```

```
root@33de406dce50:/# ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.1  0.0   4588  3840 pts/0    Ss   20:38   0:00 /bin/bash
root          11  0.0  0.1   7888  3968 pts/0    R+   20:39   0:00 ps aux
root          12  0.0  0.0   2708  1280 pts/0    S+   20:39   0:00 head
root@33de406dce50:/# |
```

A Docker container was launched using an Ubuntu base image. Unlike virtual machines, containers share the host operating system kernel and provide lightweight isolation. The container demonstrated significantly lower overhead compared to a full VM environment (WSL2), especially in terms of process isolation and system resource usage.

Containers are more lightweight and efficient, while VM-like systems provide stronger isolation but use more resources.

Part B

Understand cloud virtualization and hypervisor-based infrastructure.

My Amazon Web Services account was suspended, so I could not access EC2 instances for this lab. To complete the required tasks, I used Google Cloud Shell as an alternative cloud environment. AWS normally provides virtual machines through services like EC2, which run on virtualized infrastructure and allow users to monitor resources and execute applications. This functionality was replicated using the alternative setup.

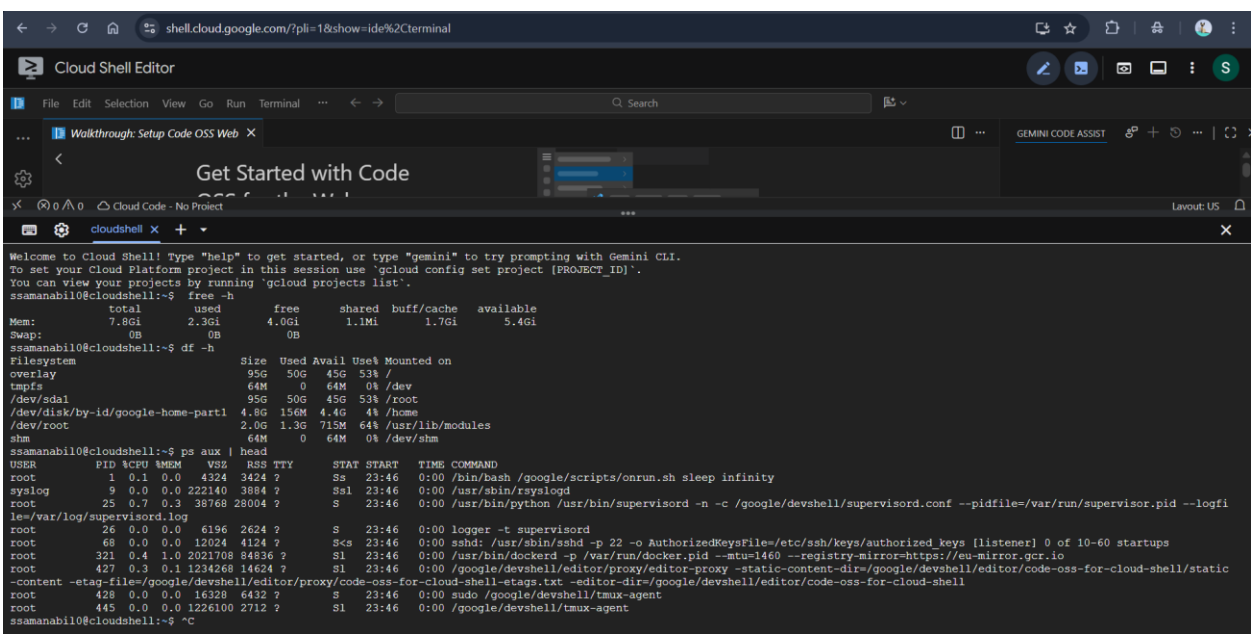
aws

Amazon Web Services Sign In

Authentication failed because your account has been suspended.

If you believe your account was suspended due to non-payment of outstanding balance due on your account, you can pay now using the Payments to reactivate your account. If you do not pay or provide a payment method to resolve your outstanding balance, your account resources may be terminated.

If your account was suspended for reasons other than non-payment of outstanding dues, contact AWS customer support Contact Us



Part C

venv activated

```
sama@DESKTOP-L3NT4QA:~$ python3 -m venv myenv
sama@DESKTOP-L3NT4QA:~$ source myenv/bin/activate
(myenv) sama@DESKTOP-L3NT4QA:~$ pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_
 (2.7 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 103.4/103.4 kB 1.0 MB/s eta 0:00:00
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.3-py3-none-any.whl (110 kB)
```

Flask install success

```
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 103.4/103.4 kB 1.0 MB/s eta 0:00:00
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.3-py3-none-any.whl (110 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 110.5/110.5 kB 3.7 MB/s eta 0:00:00
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 134.9/134.9 kB 1.3 MB/s eta 0:00:00
Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 226.5/226.5 kB 1.0 MB/s eta 0:00:00
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.3.3 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3
.1.8
```

Flask running

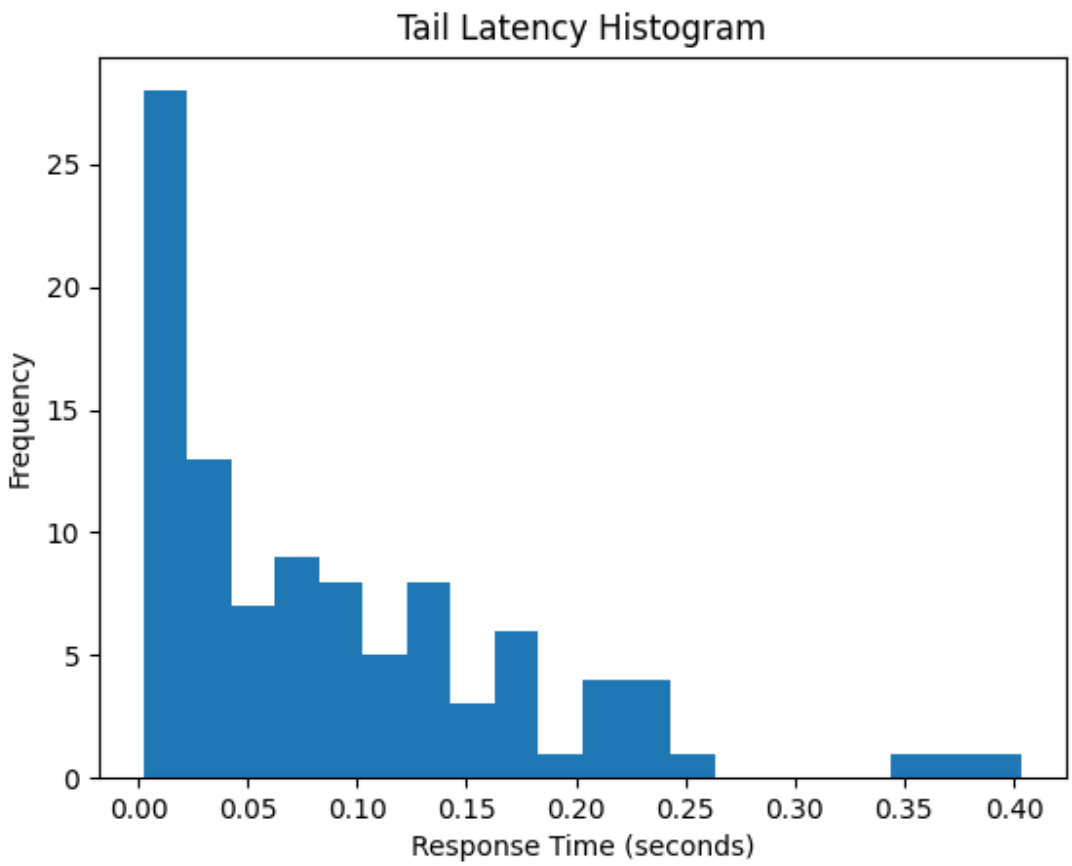
```
(myenv) sama@DESKTOP-L3NT4QA:~$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.21.49.17:5000
Press CTRL+C to quit
```

ApacheBench output

```
sama@DESKTOP-L3NT4QA:~$ ab -n 100 -c 10 http://127.0.0.1:5000/
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 127.0.0.1 (be patient).....done


Server Software:        Werkzeug/3.1.8
Server Hostname:        127.0.0.1
Server Port:            5000

Document Path:          /
Document Length:        27 bytes

Concurrency Level:      10
Time taken for tests:   1.543 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      20000 bytes
HTML transferred:       2700 bytes
Requests per second:    64.83 [#/sec] (mean)
Time per request:       154.257 [ms] (mean)
Time per request:       15.426 [ms] (mean, across all concurrent requests)
Transfer rate:          12.66 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   1.2      0      12
Processing:    12  113 106.0     77     578
Waiting:        2  103 105.9     67     567
Total:         12  114 105.9     77     578

Percentage of the requests served within a certain time (ms)
  50%     77
  66%    129
  75%    162
  80%    193
  90%    231
  95%    339
  98%    532
  99%    578
 100%    578 (longest request)
sama@DESKTOP-L3NT4QA:~$
```

histogram

```
(myenv) sama@DESKTOP-L3NT4QA:~$ ls
app.py  app.py.save  hist.py  histogram.png  myenv
(myenv) sama@DESKTOP-L3NT4QA:~$ explorer.exe .
(myenv) sama@DESKTOP-L3NT4QA:~$
```



histogram was saved as a PNG file because WSL does not support graphical display windows.

Which is better for microservices: VM or container?

AWS Nitro splits responsibilities into hardware-based components to improve security, performance, and isolation. By offloading virtualization functions to dedicated hardware, it reduces the attack surface and improves efficiency

Why AWS splits Nitro into hardware components?

VMs are preferred when strong isolation is required, such as running different operating systems, handling sensitive workloads, or ensuring strict security boundaries between applications.

When to use VMs over containers?

VMs are preferred when strong isolation is required, such as running different operating systems, handling sensitive workloads, or ensuring strict security boundaries between applications.

How does tail latency change with parallel calls?

Tail latency increases as the number of parallel requests increases due to resource contention, queueing delays, and uneven processing times, causing some requests to experience significantly higher response times.